

TP8 Java Yassine Hamrouni

Exercice 1 - Les listes chaînées

1- Créer un record `Link` dans le paquetage `fr.uge.data` correspondant à un maillon de la liste chaînée stockant des entiers.

En aucun cas, l'utilisateur de la classe ne devra lui-même manipuler des maillons.

Quelle doit être la visibilité du record `fr.uge.data.Link` ?

Écrire un main de test dans cette classe créant deux maillons contenant les valeurs 13 et 144.

La visibilité du record `Link` doit être `package-private` (default) pour restreindre l'accès.

2- Créer une classe `fr.uge.data.LinkedList` qui permettra de manipuler une liste chaînée par son premier maillon, avec :

une méthode `add(int value)` qui ajoute un élément en tête de la liste.

```
public void add(int value) {  
    link = new Link(value, link);  
}
```

une méthode `get(index)` qui renvoie l'élément à l'index (en commençant à 0).

Comment faire en sorte que le code qui vérifie que l'index est valide soit en $O(1)$?

Faites les changements qui s'imposent.

```
public int get(int index) {  
    if (index < 0 || index >= size) {  
        throw new IndexOutOfBoundsException("Index: " + index + ", Size: " + size);  
    }  
    Link current = link;  
    for (int i = 0; i < index; i++) {  
        current = current.next();  
    }  
    return current.value();  
}
```

3- une méthode `forEach(lambda)` qui appelle la lambda avec la valeur de chaque maillon de la liste.

```
public void forEach(Consumer<Integer> lambda) {
    Objects.requireNonNull(lambda);
    Link current = link;
    while (current != null) {
        lambda.accept(current.value());
        current = current.next();
    }
}
```

4- une méthode `toString` qui représente la liste chaînée avec des “->” entre les valeurs.

```
@Override
public String toString() {
    StringBuilder sb = new StringBuilder();
    Link current = link;
    while (current != null) {
        sb.append(current.value());
        if (current.next() != null) {
            sb.append(" --> ");
        }
        current = current.next();
    }
    return sb.toString();
}
```

Exercice 2 - Liste chaînée (suite)

Dans le but de pouvoir ré-utiliser la liste dans différents codes, changer les classes `fr.uge.data.LinkedList` et `fr.uge.data.Link` pour une implantation plus générique à base d’Object.

```
package fr.uge.data;

record Link(Object value, Link next) {

}
```

```
package fr.uge.data;

import java.util.Objects;
import java.util.function.Consumer;
```

```

public class LinkedList {
    private Link link;
    private int size;

    public void add(Object value) {
        link = new Link(value, link);
        this.size++;
    }

    public Object get(int index) {
        if (index < 0 || index >= size) {
            throw new IndexOutOfBoundsException("Index: " + index + ", Size: " + size);
        }
        Link current = link;
        for (int i = 0; i < index; i++) {
            current = current.next();
        }
        return current.value();
    }

    public void forEach(Consumer<Object> lambda) {
        Objects.requireNonNull(lambda);
        Link current = link;
        while (current != null) {
            lambda.accept(current.value());
            current = current.next();
        }
    }

    @Override
    public String toString() {
        StringBuilder sb = new StringBuilder();
        Link current = link;
        while (current != null) {
            sb.append(current.value());
            if (current.next() != null) {
                sb.append(" --> ");
            }
            current = current.next();
        }
        return sb.toString();
    }
}

```

Dans la classe Main, expliquer pourquoi le code suivant ne fonctionne pas

Car notre Consumer prend un argument , par conséquent on peut utiliser la méthode length(). Il faudrait alors caster en String.

```
var list = new LinkedList();
list.add("hello");
list.add("world");

list.forEach(s -> {
    String str = (String) s; // Casting explicite
    System.out.println("string " + str + " length " + str.length());
});
```

Exercice 3 - Générification de Linkedlink

1- Rappeler quel est l'intérêt d'utiliser un type paramétré ici ?

Eviter le casting.

2- Paramétrer la classe fr.uge.data.LinkedList pour que celle-ci soit générique.

```
package fr.uge.data;

import java.util.function.Consumer;

public class LinkedList<T> {
    private Link<T> link;
    private int size;

    public void add(T value) {
        link = new Link<>(value, link);
        size++;
    }

    public T get(int index) {
        if (index < 0 || index >= size) {
            throw new IndexOutOfBoundsException("Index: " + index + ", Size: " + size);
        }
        Link<T> current = link;
        for (int i = 0; i < index; i++) {
            current = current.next();
        }
        return current.value();
    }
}
```

```

    public void forEach(Consumer<T> lambda) {
        Link<T> current = link;
        while (current != null) {
            lambda.accept(current.value());
            current = current.next();
        }
    }

    @Override
    public String toString() {
        StringBuilder sb = new StringBuilder();
        Link<T> current = link;
        while (current != null) {
            sb.append(current.value());
            if (current.next() != null) {
                sb.append(" --> ");
            }
            current = current.next();
        }
        return sb.toString();
    }
}

```

3- Modifier la classe fr.uge.data.main.Main en conséquence.

```

package fr.uge.data.main;

import fr.uge.data.LinkedList;

public class Main {
    public static void main(String[] args) {
        LinkedList<String> list = new LinkedList<>();
        list.add("hello");
        list.add("world");

        list.forEach(s -> System.out.println("string " + s + " length " + s.length()));
    }
}

```

4- écrire une méthode `removeIf` qui supprime tous les éléments vrai pour un prédicat.

```
public void removeIf(Predicate<T> predicate) {
    while (link != null && predicate.test(head.value())) {
        link = link.next();
        size--;
    }

    Link<T> current = link;
    while (current != null && current.next() != null) {
        if (predicate.test(current.next().value())) {
            current.next = current.next().next();
            size--;
        } else {
            current = current.next();
        }
    }
}
```